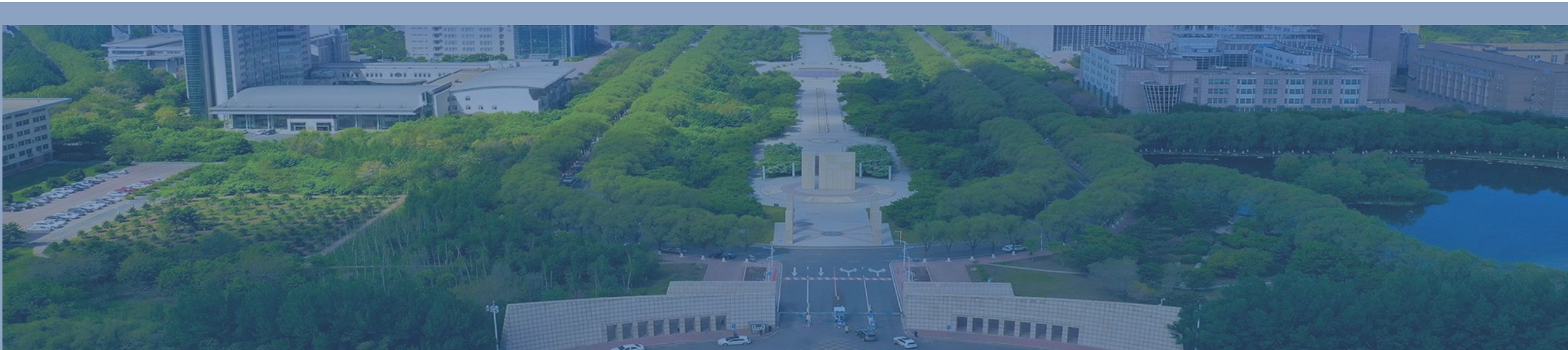


吉林大学



第四章 分治法作业





1 求解①和②情况下的下列递归关系式，并简单分析：

$$T(n) = \begin{cases} g(n) & n \text{ 足够小} \\ 2T(n/2) + f(n) & \text{否则} \end{cases}$$

① $g(n)=O(1)$, $f(n)=O(n)$;

② $g(n)=O(1)$ 和 $f(n)=O(1)$ 。

2 根据4.2节开始所给出的二分查找策略，写一个二分查找的递归过程。

3 设计一个“三分”查找算法，它首先检查 $n/3$ 处的元素是否等于某个 x 的值，然后检查 $2n/3$ 处的元素。这样，或者找到 x ，或者把集合缩小到原来的 $1/3$ 。分析算法在各种情况下的计算复杂度。

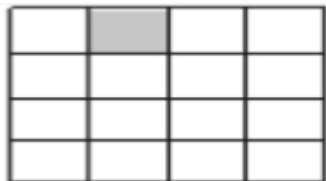
4 实现“自底向上”的归并排序算法,从而取消对栈空间的利用。

5 二维极大点问题，采用分治策略求解，给出下面实例的求解过程及结果：

(1,2) (2,8) (3,5) (4,1) (5,4) (6,7) (7,6) (8,3)。

6 N 个硬币放袋子里，其中一枚是假币，并且假币比真币轻，设计一个算法找到那枚假币。要求写明算法思想，并给出算法描述（注意书写规范）。

7 有一个 $2^k \times 2^k$ 的方格棋盘，恰有一个方格是黑色的，其他为白色。你的任务是用包含3个方格的L型牌覆盖所有白色方格，且黑色方格不能被覆盖，任意一个白色方格不能同时被两个或更多牌覆盖。如图所示为L型牌的4种旋转方式。请设计算法，解决棋盘覆盖问题。要求：写清算法思路，感兴趣的同学鼓励上机编程实现。



1、求解①和②情况下的下列递归关系式，并简单分析：

$$T(n) = \begin{cases} g(n) & n \text{ 足够小} \\ 2T(n/2) + f(n) & \text{否则} \end{cases}$$

① $g(n)=O(1)$ 和 $f(n)=O(n)$ ；

② $g(n)=O(1)$ 和 $f(n)=O(1)$ 。



①当 $g(n)=O(1)$ 和 $f(n)=O(n)$ 时

不妨设 $g(n)=a$ ， $f(n)=bn$ ， a 、 b 为常数，
令 $n=2^k$ ，则：

$$T(n)=2T(n/2)+bn$$

$$= 4T(n/4)+2bn$$

$$= \dots \dots$$

$$= 2^k T(n/2^k) + kbn$$

$$= an + bn \log_2 n = O(n \log_2 n)$$





②当 $g(n)=O(1)$ 和 $f(n)=O(1)$ 时,
不妨设 $g(n)=c$, $f(n)=d$, 令 $n=2^k$, 则:

$$\begin{aligned}T(n) &= 2T(n/2) + d \\&= 4T(n/4) + 2d \\&= \dots \dots \\&= 2^k T(n/2^k) + kd \\&= cn + d \log_2 n \\&= O(n)\end{aligned}$$

本题满足主定理第一种情况, 上题是第二种情况。 ($a=1$, $b=2$)

• 主定理: 设 $a \geq 1, b > 1$ 为常数, $f(n)$ 为函数, $T(n)$ 为非负整数, 且
 $T(n) = aT(n/b) + f(n)$, 则 $T(n)$ 有如下渐近界:

1. 若 $\exists$ 常数 $\varepsilon > 0$, 使得 $f(n) = O(n^{\log_b a - \varepsilon})$, 则 $T(n) = \Theta(n^{\log_b a})$

$f(n)$ 的数量级低于 $n^{\log_b a}$

2. 若 $f(n) = \Theta(n^{\log_b a})$, 则 $T(n) = \Theta(n^{\log_b a} \log n)$

$f(n)$ 的数量级等于 $n^{\log_b a}$

3. 若 $\exists$ 常数 ε , 使得 $f(n) = \Omega(n^{\log_b a + \varepsilon})$,
且 $\exists$ 常数 $c < 1$, 且 n 足够大时, 有 $af(n/b) \leq cf(n)$,
则 $T(n) = \Theta(f(n))$

$f(n)$ 的数量级高于 $n^{\log_b a}$

2、根据4.2节开始所给出的二分查找策略，写一个二分查找的递归过程。





Procedure BINSRCH2(A, low, high, x, j)

integer mid;

if $low \leq high$ **then**

$mid \leftarrow \lfloor (low + high) / 2 \rfloor$

case

: $x = A(mid)$: $j \leftarrow mid$; **return**

: $x > A(mid)$: **BINSRCH2**(A, $mid + 1$, high, x, j)

: $x < A(mid)$: **BINSRCH2**(A, low, $mid - 1$, x, j)

endcase

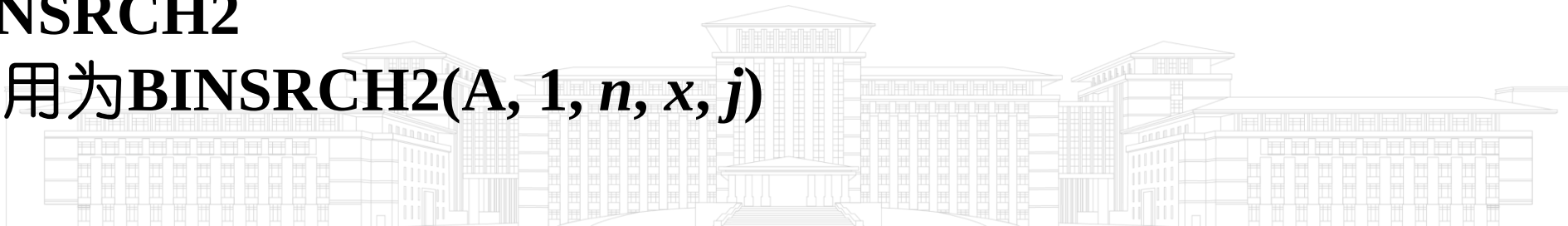
else

$j \leftarrow 0$;

endif

end BINSRCH2

初次调用为 **BINSRCH2**(A, 1, n, x, j)



3、作一个“三分”查找算法，它首先检查 $n/3$ 处的元素是否等于某个 x 的值，然后检查 $2n/3$ 处的元素。这样，或者找到 x ，或者把集合缩小到原来的 $1/3$ 。分析算法在各种情况下的计算复杂度。



Procedure ThriSearch(A, n, x, j)

integer $low, high, p1, p2$

$low \leftarrow 1; high \leftarrow n$

while $low \leq high$ **do**

$p1 \leftarrow \lfloor (high + 2low) / 3 \rfloor$

$p2 \leftarrow \lfloor (2high + low) / 3 \rfloor$

case

$:x = A(p1): j \leftarrow p1; \text{return}$

$:x = A(p2): j \leftarrow p2; \text{return}$

$:x < A(p1): high \leftarrow p1 - 1$

$:x > A(p2): low \leftarrow p2 + 1$

$: \text{else}: low \leftarrow p1 + 1; high \leftarrow p2 - 1$

endcase

repeat

$j \leftarrow 0$

end ThriSearch

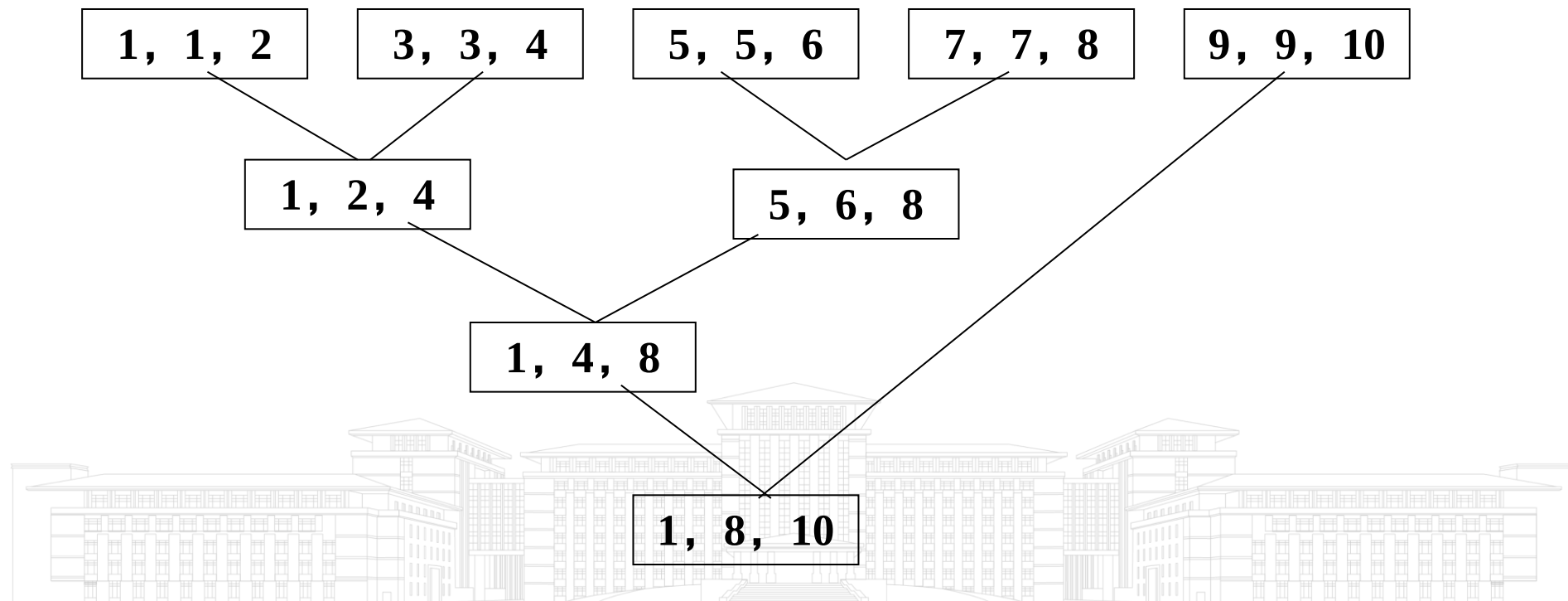


三分查找的时间复杂度

成功情况下		
最好	最坏	平均
$O(1)$	$O(\log_3(n))$	$O(\log_3(n))$
失败情况下		
最好	最坏	平均
$O(\log_3(n))$	$O(\log_3(n))$	$O(\log_3(n))$

4、写一个“由底向上”的归并分类算法，从而取消对栈空间的利用。

实例：1, 2, 3, 4, 5, 6, 7, 8, 9, 10



Procedure MERGESORT1(n)

integer low , mid , $high$, $step$, k

$step \leftarrow 1$; $k \leftarrow \lfloor \log n \rfloor$

while $step \leq 2^k$ do

$low \leftarrow 1$

 while $low + step - 1 < n$ do

$mid \leftarrow low + step - 1$;

$high \leftarrow \min\{mid + step, n\}$;

 MERGE (low , mid , $high$);

$low \leftarrow high + 1$;

 repeat

$step \leftarrow step \times 2$;

repeat

end MERGESORT1



5、二维极大点问题实例

采用分治策略求解二维极大点问题，给出下例求解过程及求解结果。

实例：求下列点中的极大点

$(1,2)(2,8)(3,5)(4,1)(5,4)(6,7)(7,6)(8,3)$

理解算法，理解算法执行过程；

理解递归执行过程及执行结果。



首先将所有的点按 x 值排序（如果 x 相等则再按 y 排序） $\leftarrow$

可得如下结果： $\leftarrow$

(1,2) (2,8) (3,5) (4,1) (5,4) (6,7) (7,6) (8,3) $\leftarrow$

①分：找出垂直于 x 轴的中位线 l ，将整个点集分为两个子集 S_L 和 S_R $\leftarrow$

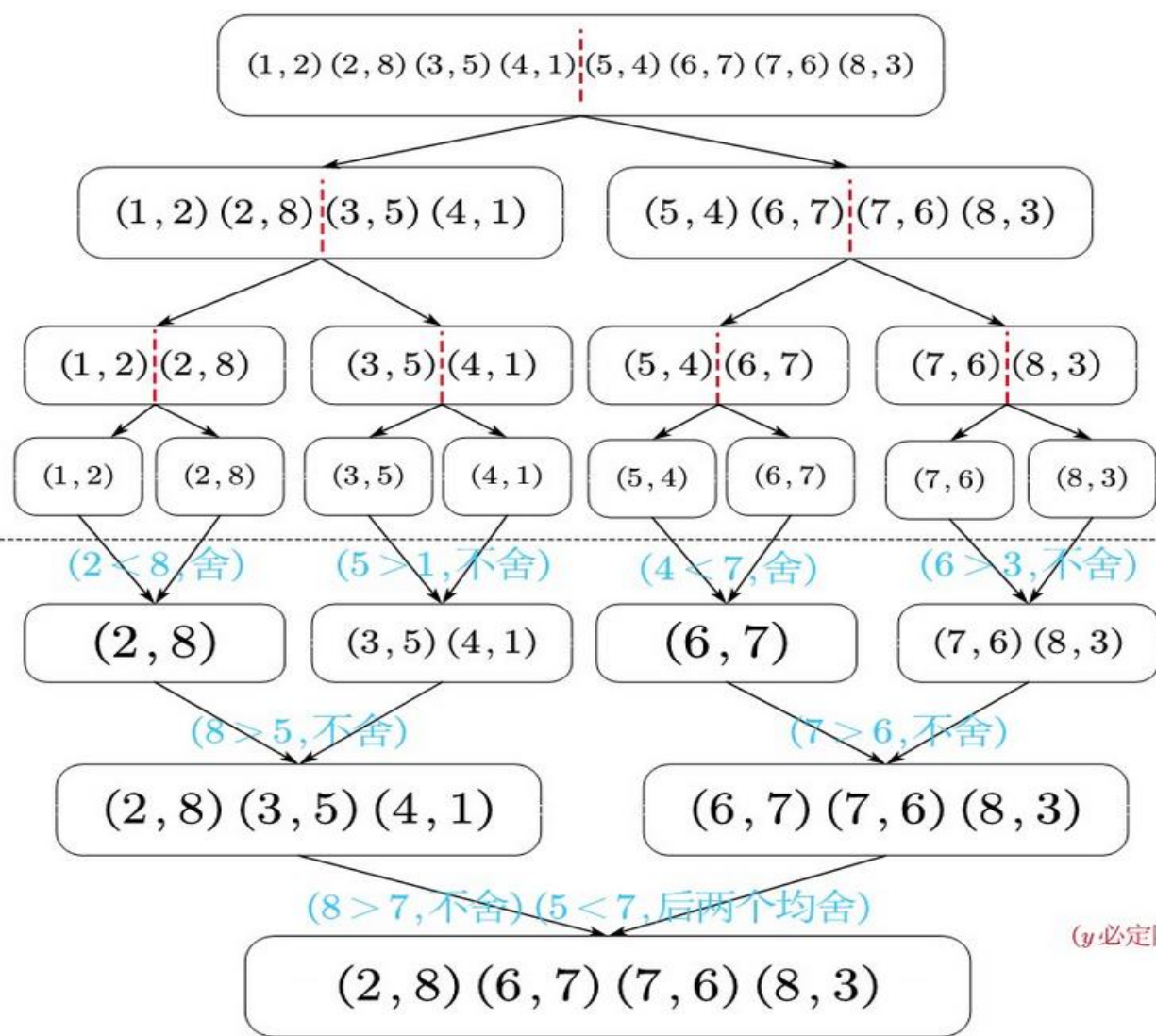
②治：分别找出 S_L 和 S_R 的极大点。 $\leftarrow$

③合：对于 S_L 中的极大点 p ，如果 p_y 小于 S_R 中任一极大点的 y 值，则 p 被支配，舍弃掉。

可得如下过程： $\leftarrow$



分治过程



合过程

(y 必定降序排列, 仅比较最大即可)

注意, 在合并过程要让 S_L 中的极大点 p 的 y 值大于 S_R 中任一极大点的 y 值, 即大于 S_R y 值的最大值, 而在归并过程中, x 升序保证了 y 值降序才能保留, 所以合并时, 只需让左侧的值依次比较右侧的第一个极大点的 y 值即可, 更进一步, 左侧的 y 值也是降序, 相当于找一个最小的左侧的 y 大于右侧的第一个极大点的 y 值, 可以使用二分法优化。

6、N个硬币放袋子里，其中一枚是假币，并且伪造的硬币比真币轻，设计一个算法找到那枚假币。要求写明算法思想，并给出算法描述（注意书写规范）。

算法思想：

N如果是偶数，就分为两半，每半 $N/2$ 枚，称重较轻的继续二分，直到每半只有一枚，较轻者为假币；

N如果是奇数，拿出一枚，分为两半，若不一样重，就继续选轻的二分，若一样重，则拿出来那枚就是假币。



算法描述:

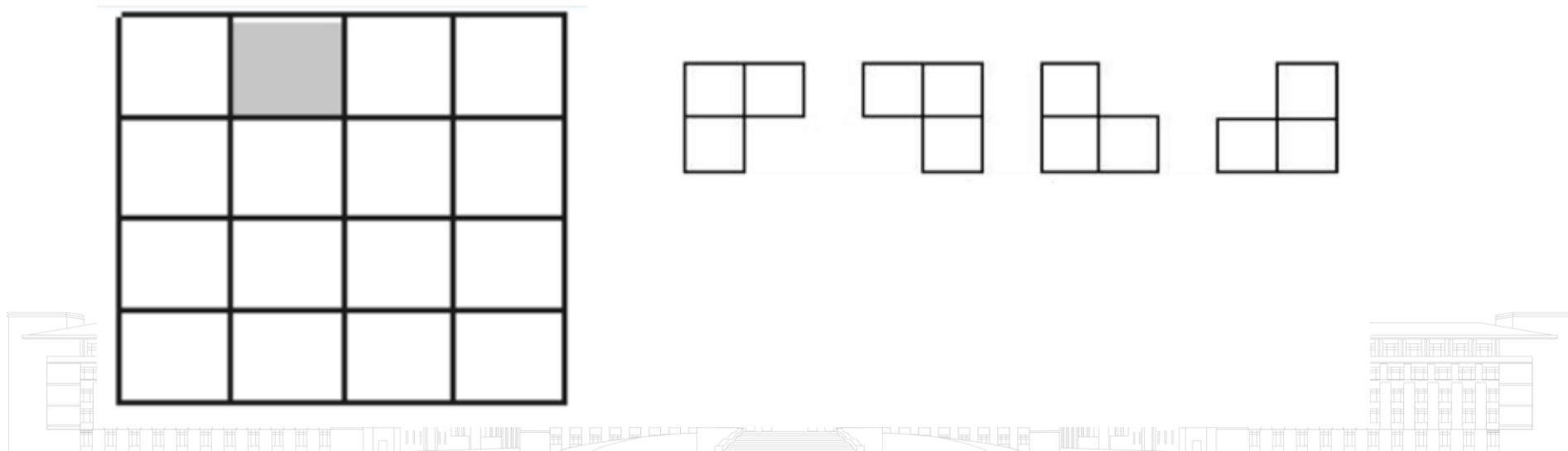
```
procedure FAKE(c)
  if sizeof(c)==1 then return c; //递归出口
  if sizeof(c)%2==0
    then
      (c1,c2)=partition(c);
      lighter=cmp(c1,c2);
      FAKE(lighter);
    else  c0=takeone(c);
          (c1,c2)=partition(c-c0);
          if cmp(c1,c2)==0 then return c0; //递归出口
            else lighter=cmp(c1,c2);  FAKE(lighter);
          endif
    endif
end
```

时间复杂度分析

$$T(n) = T(n/2) + 1$$

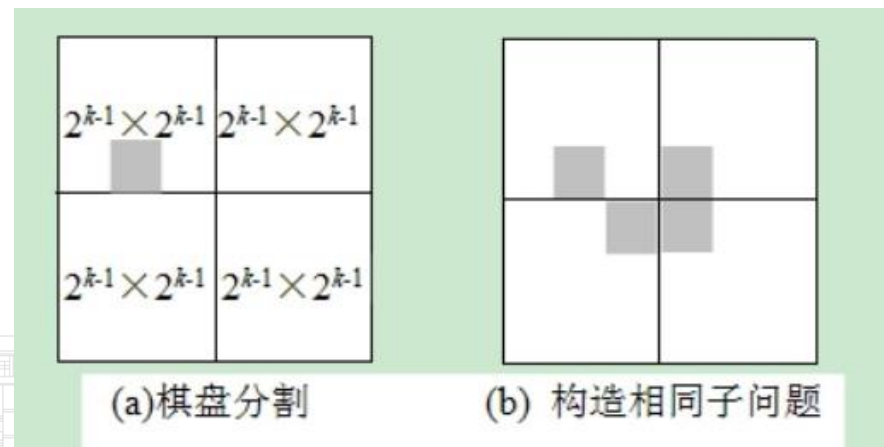
$$T(n) = O(\log n)$$

7、有一个 $2^k \times 2^k$ 的方格棋盘，恰有一个方格是黑色的，其他为白色。你的任务是用包含3个方格的L型牌覆盖所有白色方格，且黑色方格不能被覆盖，任意一个白色方格不能同时被两个或更多牌覆盖。如图所示为L型牌的4种旋转方式。请设计算法，解决棋盘覆盖问题。要求：写清算法思路，感兴趣的同学鼓励上机编程实现。



算法思路

- 1) **分**: 原始的棋盘划分为四个区域——左上、右上、左下、右下; 判断黑格所在区域, 其它三个区域, 在中间用L形牌填充; 这样把原问题拆分成了缩小的规模的子问题。
- 2) **治**: 递归调用——分别对四个区域进行递归调用; 递归出口: 当每个区域只包含四个单元格时, 直接处理——除黑色单元格外, 其它三格用L牌填充。
- 3) **合**: 空操作。



下面讨论棋盘覆盖问题中数据结构的设计。

(1) 棋盘：可以用一个二维数组`board[size][size]`表示一个棋盘，其中， $\text{size}=2^k$ 。为了在递归处理的过程中使用同一个棋盘，将数组`board`设为全局变量；

(2) 子棋盘：整个棋盘用二维数组`board[size][size]`表示，其中的子棋盘由棋盘左上角的下标`tr`、`tc`和棋盘大小`s`表示；

(3) 特殊方格：用`board[dr][dc]`表示特殊方格，`dr`和`dc`是该特殊方格在二维数组`board`中的下标；

(4) L型骨牌：一个 $2^k \times 2^k$ 的棋盘中有一个特殊方格，所以，用到L型骨牌的个数为 $(4^k - 1) / 3$ ，将所有L型骨牌从1开始连续编号，用一个全局变量`t`表示。

设全局变量`t`已初始化为0，分治法求解棋盘覆盖问题的算法用C++语言描述。





本章作业结束

